Theoretical Computer Science Toolkit

Hector Jimenez

July 23, 2026

1 Preliminaries

1.1 Big O and friends

Recall that $f(x) = O(g(x))$ $(as\ x \to \infty) \iff \exists c, x_0\ s.t.\ |f(x)| \leq c \cdot g(x),\ \forall x \geq x_0$. **Example:** $\sum_{i=1}^{n} i = \frac{n(n+1)}{2} = \frac{n^2}{2} + \frac{n}{2} = O(n^2) = \frac{n^2}{2} + O(n)$.

Definition 1.1 (Harmonic Number)**.** *Let $n \in \mathbf{N}$, then we define $H_n = 1 + 1/2 + 1/3 + 1/4 + ... + 1/n$.*

We can get an **upper bound** and a **lower bound** for harmonic number in the following way:

$$\begin{aligned} H_n = 1 + 1/2 + 1/3 + 1/4 + ... + 1/n \leq \\ 1 + 1/2 + 1/2 + 1/4 + 1/4 + 1/4 + 1/4 + 1/8 + ... \leq \\ \lfloor \log_2 n \rfloor + 1. \end{aligned}$$

$$\begin{aligned} H_n = 1 + 1/2 + 1/3 + 1/4 + ... + 1/n \geq \\ 1 + 1/2 + 1/4 + 1/4 + 1/8 + 1/8 + 1/8 + 1/8 + 1/16... \geq \\ 1/2\lceil \log_2 n \rceil + 1. \end{aligned}$$

Since that $H_n \leq O(\log n)$ and $H_n \geq \Omega(\log n)$, we can conclude that $H_n = \Theta(\log n)$.

Definition 1.2 ($f(x) \sim g(x)$)**.** *We say that $f(x) \sim g(x)$ as $x \to \infty$ if $\frac{f(x)}{g(x)} \to 1$.*

We recall the definition of a Taylor Series:

Theorem 1.1 (Taylor Series)**.** *If $f(x)$ is analytic at $x = a$, then it can be represented by its Taylor series:*

$$f(x) = \sum_{n=0}^{\infty} \frac{f^{(n)}(a)}{n!}(x-a)^n,$$

which converges to $f(x)$ for all x within the radius of convergence.

Definition 1.3 (e^x)**.** *We define the function $e^x = \sum_{i=0}^{\infty} \frac{x^n}{n!},\ \forall x \in \mathbf{R}$.*

For example, the function $\ln 1 + x = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n!}$, for $|x| \leq 1$. Additionally, we can use this equivalence to approximate $\ln 1 + x = \sum_{n=0}^{\infty} (-1)^{n-1} \frac{x^n}{n!} \sim x$ when $x \to 0^+$. We can also exponent this and get: $\exp x \sim 1 + x$.

Definition 1.4 (Central Binomial Coefficient)**.** *We define the central binomial coefficient as $C_n := \binom{n}{n/2}$. We notice that the probability of getting half of n flipped coins is $\frac{C_n}{2^n}$.*

Proposition 1.2. $C_n = \Theta(\frac{2^n}{\sqrt{n}})$.

Proof. We can bound above and below the expression. Lets do a simple example to see this. $C_{10} = \frac{10 \cdot 9 \cdot 8 \cdots 2 \cdot 1}{5 \cdot 4 \cdots 1 \cdot 5 \cdot 4 \cdots 1}$. Then we divide by 2^n and get: $C_{10}/2^n = \frac{10 \cdot 9 \cdot 8 \cdots 2 \cdot 1}{10 \cdot 8 \cdots 2 \cdot 10 \cdot 8 \cdots 2}$. Furthermore we can potentiate to $()^2$ and get: $(\frac{C_n}{2^n})^2 = \frac{10 \cdot 10 \cdot 9 \cdot 9 \cdot 8 \cdot 8 \cdots 2 \cdot 2 \cdot 1}{10 \cdot 10 \cdot 10 \cdot 10 \cdot 8 \cdot 8 \cdot 8 \cdot 8 \cdots 2 \cdot 2 \cdot 2 \cdot 2} = \frac{9 \cdot 9 \cdot 7 \cdot 7 \cdot 5 \cdot 5 \cdot 3 \cdot 3}{10 \cdot 10 \cdot 8 \cdot 8 \cdot 6 \cdot 6 \cdot 4 \cdot 4 \cdot 2 \cdot 2}$. From this we can get an upper bound: $(\frac{C_n}{2^n})^2 \leq \frac{10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1}{11 \cdot 10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2} = \frac{1}{11} = \frac{1}{n+1}$. Also we can get a lower bound: $(\frac{C_n}{2^n})^2 \geq \frac{9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2}{10 \cdot 9 \cdot 8 \cdot 7 \cdot 6 \cdot 5 \cdot 4 \cdot 3 \cdot 2 \cdot 1} = \frac{1}{10} = \frac{1}{n}$. □

1.2 Sum of Random Variables, Gaussian and Central Limit Theorem

We work with the following setup: let $X_1, X_2, ..., X_n$ be i.i.d random variables with $\mathbf{P}[X_i = 1] = p$ and $\mathbf{P}[X_i = 0] = 1 - p = q$. Let $S_n = X_1 + X_2 + ... + X_n$, then $\mathbf{E}(S_n) = \mathbf{E}(X_1) + ... + \mathbf{E}(X_n) = pn$.

2 Boolean Functions

A boolean function is perhaps the most simplest function in computer science.

Definition 2.1 (Boolean Function)**.** *A function $f : \{0,1\}^n \to \{0,1\}$ is called a boolean function.*

There are different ways to measure the complexity of a boolean function. Here we present four: **decision tree complexity, sensitivity, certificate complexity and degree**.

Definition 2.2 (Decision Tree complexity)**.** *The decision tree complexity $d(f)$ of a boolean function is the depth of the smallest tree that computes a boolean function.*

Definition 2.3 (Sensitivity)**.** *The hypercube is a graph that connects inputs of a boolean function f which differ by 1 bit. The nodes (inputs) in the hypercube are colored with the output of the boolean function. The sensitivity complexity $s(f)$ is defined as the maximum number of neighbours with different color in the hypercube.*

Definition 2.4 (Certificate Complexity)**.** *A certificate is a set of coordinates in the input that once we fixed those coordinates, the value of the function is determined. The certificate complexity $c(f)$ is the smallest number such that every input x has a certificate of size $c(f)$.*

Definition 2.5 (Degree Complexity)**.** *Any function $f : \{0,1\}^n righarrow\mathbb{R}$ can be expressed as a multilinear polynomial. The degree complexity $deg(f)$ of a boolean function is the degree of this polynomial.*

We can stablish bound between these complexity mesasures:

Proposition 2.1. *The following bounds hold:*

- $s(f) \leq c(f)$
- $c(f) \leq d(f)$
- $deg(f) \leq d(f)$
- $d(f) \leq c(f)^2$

3 Fourier Analysis of Boolean Functions

Let start motivating the topic constructing a polynomial that represents a boolean function.

Example 3.1 (Fourier representation of Max boolean function)**.** *Let* $Max_2 : \{-1,1\}^2 \to \{-1,1\}$*, which takes the max of the input. We construct the fourier representation in the following way:* $Max_2(x_1, x_2) = \frac{1}{2} + \frac{1}{2}x_1 + \frac{1}{2}x_2 + \frac{1}{2}x_1x_2$.

Theorem 3.1. *Any function* $f : \{+1,-1\}^n \to \mathbb{R}$ *is uniquely expressible as a multilinear polynomial.*

We represent function as $f(x) = f(x_1, ..., x_n) = \sum_{S\subseteq[n]} \hat{f}(S) \prod_{i\in S} x_i$. In this way we can see that the fourier expansion is a linear combinations "XOR"s functions. This is because $\prod_{i\in S} x_i$ is the parity or XOR of bits in S.

4 Circuits

Definition 4.1 (Circuit)**.** *A circuit is a Directed Acyclic Graphs (DAGs) that computes Boolean Functions* $f : \{0,1\}^n \to \{0,1\}$*. The nodes of a circuit are called gates. A circuit can be seen as a tree, where the leafs are input gates that recieves an input* $x = (x_1, x_2, ..., x_n)$ *and the root generates an output bit.*

Definition 4.2 (Size and Depth of Circuit)**.** *The Size and Depth of a circuit* C *are defined as the number of non-input gates and the longest input-output path, respectively.*

Example 4.1. *Some common basis are the following:* $\mathcal{B}_1 = \{\neg, binary \vee, binary \wedge\}, \mathcal{B}_2 = \{all\ g : \{0,1\}^2 \to \{0,1\}\}, \mathcal{B}_3 = \{\neg, all\ fan\text{-}in \vee, all\ fan\text{-}in \wedge\}$*. Notice that* $\mathcal{B}_3$ *allow for constant depth computations. For example, we can add two n-bits numbers in constant depth. Instead for* B_1*, we need at least a depth of* $\log n$ *to look all bits.*

Example 4.2. *We study the function* ***AND****:* $\{0,1\}^n \to \{0,1\}$*. Notice that with* $\mathcal{B}_3$ *we can generate a circuit with size 1. Instead with* $\mathcal{B}_2$ *we can generate a circuit with size* $O(n)$ *and depth* $O(\log n)$*.*

Example 4.3. *We study the* ***Palindrome*** *function* $p : \{0,1\}^{2n} \to \{0,1\}$*. With* $\mathcal{B}_3$ *we can generate a circuit with depth 2, by generating an "equal" gate.*

Definition 4.3 (Basis)**.** *We call* $\mathcal{B}(C)$ *the basis of a circuit* C*, which is a set of allowed operations in gates of a circuit* C*.*

We have talk about constant size inputs of size n, this mean that for each input size we need to generate a different circuit. In order to accept a certain language L, we define a **circuit family** $(C_n)_{n=1}^{\infty}$, where the circuit C_n should accepts all words in $L \cap \{0,1\}^n$. Now, we define some families of languages that are categorized by the size of circuit families (with $\mathcal{B}_3$) that accept them.

Definition 4.4 (Family AC^0)**.** *The collection of languages decided by a circuit family of depth* $O(1)$ *and size* $poly(n)$*.*

Definition 4.5 (Family NC)**.** *The collection of languages decided by a circuit family of depth* $polylog(n)$ *and size* $poly(n)$*.*

There are another notations about language decidable by circuits.

Definition 4.6 (P-Poly)**.** *The family of languages decidable by poly(n)-size circuits size.*

A cool proposition about circuits.

Proposition 4.1 (Has 86)**.** *There exists a language* $L \in P$ *with no poly-size, constant-depth circuits.*

Proposition 4.2 (CLIQUE)**.** *CLIQUE requires exponential size AND/OR circuits.*

5 Communication Complexity

We study the case of 2-party communication between Alice and Bob. The idea is that A and B want to compute the function $f : X \times Y \to Z$. The goal is given $x \in X$ (Alice) and $y \in Y$ (Bob) that both Alice and Bob computes $f(x, y)$. We only care about how many communication steps Alice and Bob perform. We consider that each time Alice and Bob communicate to each other a cost of 1-bit.

Example 5.1 (EQUALITY)**.** *Alice and Bob have both words from* $\{0,1\}^n$ *and they try to know if these are equal. So the equality can be represented by* $EQ : \{0,1\}^n \times \{0,1\}^n \to \{0,1\}$.

Proposition 5.1. $n + 1$ *bits are sufficient and necesary to solve* EQ.

Proof. There is an easy to detec wheter $X = Y$, Alice can send the n bits from her word to Bob, then Bob knows the answer and he send it to Alice. □

Example 5.2 (PARITY)**.** *Alice and Bob have both words from* $\{0,1\}^n$ *and they try to compute* $PARITY$ *function* $PARITY(x, y) = (\sum x_i + \sum y_i) \mod 2$.

Proposition 5.2. *The communication complexity of PARITY* $cc(PARITY)$ *holds:* $cc(PARITY) \leq 2$.